

LRN Azure Hosting Guide

Sep 23, 2026 · @Someone

1. Current state and goal

All LRN application workloads share one Windows Server 2022 Azure VM. The goal is to move each workload to the Azure service that matches how it behaves, then downsize and retire the VM.

Today the VM hosts, in IIS, a .NET Core web app, LRN.ReportsApi (.NET Core Web API) and the React Denial UI. It also runs six Windows Services and a Python processor started by Windows Task Scheduler. Data lives in Azure SQL Managed Instance.

Problem	Effect
Everything shares one VM	One heavy workload hurts all others. The Python processor has used about 10 GB RAM and destabilised the VM.
VM is paid 24/7	Batch jobs that run minutes per hour still need a VM sized for their peak.
Manual operations	Patching, IIS, Task Scheduler entries and service restarts are handled by hand.

2. Core concepts: image, container, registry

Docker is a packaging and runtime technology, not a hosting service like IIS. It packages an app with its runtime and dependencies into an image, and runs that image as a container on any host: laptop, VM or Azure.

```
flowchart LR
    A[Source code] --> B[Dockerfile<br/>build recipe]
    B --> C[Docker image<br/>packaged app]
    C --> D[ACR<br/>stores images]
    D --> E[Container<br/>running copy]
```

Term	Meaning	LRN example
Dockerfile	Steps to build the image: base runtime, copy files, install packages, start command	LRN.ReportsApi/Dockerfile
Image	Read-only, versioned package of app + runtime + libraries	lrn-reports-api:1.4.0
Container	A running instance of an image	The live ReportsApi process
Registry	Storage that hosts pull images from	Azure Container Registry
Host	Whatever runs the containers	Laptop, the VM, Azure Container Apps

Is a Docker image like an ISO?

Partly. Both are sealed, reusable packages, but they hold different things and run differently.

	ISO / VM disk image	Docker image
Contains	Full OS with its own kernel	App, runtime and user-space files, no kernel
Runs as	A VM that boots an OS	A container sharing the host's kernel
Start time	Minutes	Seconds
Typical size	Several GB	Tens to hundreds of MB

Better model: a Docker image is a portable, versioned package of your app and everything it needs, which any container host runs identically.

Linux and Windows images are not interchangeable. A container shares the host kernel, so a Linux image needs a Linux host. Azure Container Apps runs Linux containers only. The LRN apps are .NET Core and Python, so build Linux images (for example on mcr.microsoft.com/dotnet/aspnet), even while testing on the Windows VM.

3. The Azure services explained

Choose by the shape of the workload: does it answer requests or process a batch, does it stay running or exit, and is it a container?

3.1 Azure App Service

Managed hosting for web apps and APIs, the closest equivalent to IIS. You deploy published code or a container; Azure runs the web server, OS and patching.

Aspect	Detail
Runs	Always on, on an App Service Plan paid whether busy or idle
Deploy	dotnet publish, zip deploy, CI/CD or a container
Good for	Simplest move from IIS with no Docker learning curve
Less suited to	Short batch jobs (plan is paid all month)
LRN example	LRN.ReportsApi or the .NET Core web app

3.2 Azure Functions

Event-driven code that runs when a trigger fires: HTTP request, timer, queue message or new blob. Supports .NET and Python.

Aspect	Detail
Plans	Flex Consumption (serverless, current recommended), Premium (pre-warmed, VNet), Dedicated (App Service Plan)
Time limits	Plan-dependent; legacy Consumption stops at 10 min; HTTP calls must respond within about 230 s on every plan
Good for	Short independent units: a file arrives, a message arrives, a quick check every 15 min

Aspect	Detail
Less suited to	Existing long, memory-heavy batch programs without a redesign
LRN example	Log a new SharePoint file and queue it for processing

```
import azure.functions as func
app = func.FunctionApp()

@app.timer_trigger(schedule="0 */15 * * * *", arg_name="timer")
def check_new_files(timer: func.TimerRequest):
    ... # short unit of work, then return
```

3.3 Azure Container Apps

A managed platform that runs your container images without you managing servers or Kubernetes. It handles HTTPS ingress, scaling, revisions and restarts.

Aspect	Detail
Runs	Always on or on demand; scales on HTTP or events; can scale to zero
Plans	Consumption (per-second) or Dedicated workload profiles (reserved, incl. memory-optimised)
Replica size	Consumption: up to 4 vCPU / 8 GiB per replica
Good for	APIs, web apps and always-on workers as Linux containers
LRN example	LRN.ReportsApi pulled from ACR, HTTPS, min replicas = 1

3.4 Azure Container Apps Jobs

The batch counterpart: a job starts a container, it does its work and exits, and billing stops. It is the cloud replacement for a Task Scheduler entry.

flowchart LR

```

A[Trigger<br/>schedule / manual / event] --> B[Container starts]
B --> C[Program runs]
C --> D[Exit code 0 or error]
D --> E[Container stops<br/>billing stops]

```

Aspect	Detail
Triggers	Schedule (cron, UTC), Manual (portal/CLI/API), Event (e.g. queue message)
Controls	Replica timeout, retry limit, parallelism, CPU/memory per execution
History	Each execution recorded with status and times; logs to Log Analytics
LRN example	Python processor every 30 min; Master File Processor on its schedule

3.5 Azure Container Registry (ACR)

A private store for images; it runs nothing. Pipelines push images, and Container Apps, Jobs or App Service pull them. One registry holds all LRN images.

Tier	Storage	When to use
Basic	10 GB	Learning, a few images
Standard	100 GB	Normal production; suits LRN
Premium	500 GB	Private endpoints, geo-replication

3.6 Supporting services

Service	Role for LRN
Managed Identity	Lets each app or job reach Key Vault, ACR and SQL MI without passwords

Service	Role for LRN
Azure Key Vault	SQL connection strings, Graph/SharePoint credentials, API keys
Virtual Network	Private path to Azure SQL MI
Log Analytics / App Insights	Central logs, job history, alerts
NAT Gateway (optional)	Fixed outbound IP for firewall allow-lists

4. Side-by-side comparison

	App Service	Functions	Container Apps	Container Apps Jobs	ACR
Purpose	Host web apps / APIs	Run code on triggers	Run containers as services	Run containers as batch jobs	Store images
Runs code	Yes	Yes	Yes	Yes	No
Lifetime	Always on	Per trigger	Always on or scale to zero	Start, run, exit	n/a
Needs Docker	Optional	Optional	Yes	Yes	Stores Docker images
Long runs	Yes	Limited by plan	Yes	Yes (set timeout)	n/a
Cost when idle	Full plan	None on serverless	None at zero replicas	None	Fixed daily tier
IIS-world analogy	IIS site	Scheduled script / event hook	Containerised IIS site or Windows Service	Containerised Task Scheduler task	Folder of installers

5. Recommended hosting for each LRN workload

Web workloads go to Container Apps (or App Service), batch workloads to Container Apps Jobs, and the React UI to Static Web Apps.

LRN workload	Behaviour	Recommended	Alternative
LRN.ReportsApi (.NET Core)	Answers HTTP all day	Container Apps (Dockerfile exists)	App Service
.NET Core web app	Serves users all day	Container Apps	App Service
React Denial UI	Static files after build	Azure Static Web Apps	Container Apps + nginx
Python processor	Task Scheduler, runs then exits, memory-heavy	Container Apps Job (schedule)	Functions only if rewritten
LRN.MasterFileProcessorWorker	Scheduled batch: SharePoint files to lab DBs	Container Apps Job	Container App if it must poll continuously
Other Windows Services	Assess each	Finite: Job; continuous: Container App	Functions for small event handlers
Secrets		Key Vault + Managed Identity	
Images		One ACR (Standard)	
Database		Azure SQL MI (unchanged)	

```

flowchart TD
    U[Users] --> SWA[Static Web Apps<br/>React Denial UI]
    U --> CA[Container Apps<br/>web app + ReportsApi]
    J[Container Apps Jobs<br/>Python + Master File Proc] --> SQL[(Azure SQL MI<br/>LRNMaster + lab DBs)]
    CA --> SQL
    J --> G[Microsoft Graph<br/>SharePoint]
    ACR[Container Registry] -.images.-> CA
    ACR -.images.-> J
    KV[Key Vault] -.secrets.-> CA
    KV -.secrets.-> J

```

All apps and jobs use Managed Identity, reach SQL MI over the VNet, and log to one Log Analytics workspace with alerts on failed executions.

6. Master File Processor: which option is better

Recommendation: Azure Container Apps Job. Keep the existing .NET code, package it as a Linux container in ACR, and run it on a schedule with Managed Identity, Key Vault and VNet access to SQL MI.

LRN.MasterFileProcessorWorker reads Line Level and Claim Level files from SharePoint week folders via Microsoft Graph, builds CSVs, bulk copies them into per-lab tables (staging then swap) and logs per RunId. That is a finite scheduled batch, which is exactly what Jobs are for.

Criterion	Container Apps Job	Azure Functions	Always-on App Service / Container App
Code changes	Small: run once and exit	Large: split into short functions + orchestration	Small
Long runs and bulk copy	Suited; set replica timeout	Constrained by plan timeouts	Suited
Memory control	CPU/memory per job	Limited by plan	Shared with other apps
Cost between runs	None	None (serverless)	Paid 24/7

Criterion	Container Apps Job	Azure Functions	Always-on App Service / Container App
Run history	Built in per execution	Per invocation	Build it yourself
Verdict	Best fit	Only if redesigned	Works, costs more

Changes to make in the worker

- **Host model:** switch from Windows Service to a Generic Host console app that processes all enabled labs once, then exits 0 on success, non-zero on failure. Remove UseWindowsService and any internal timer loop.
- **Linux readiness:** replace C:\ paths with config values or /tmp; check file-name case sensitivity and Windows-only APIs.
- **Configuration:** per-lab flags and mapping JSONs ship in the image or a mounted file share; secrets come from Key Vault.
- **Authentication:** Managed Identity for Key Vault and, where possible, Microsoft Entra auth to SQL MI.
- **No overlaps:** parallelism 1 plus a database lock (sp_getapplock), since a slow run can overlap the next schedule.
- **Temp files:** container disk is ephemeral and limited; clean up per lab or mount Azure Files for large volumes.

```
az containerapp job create --name lrn-master-file-processor \
  --resource-group rg-lrn --environment lrn-env \
  --trigger-type Schedule --cron-expression "0 22 * * *" \
  --image lrnregistry.azurecr.io/lrn-master-file-proc:3.2.0 \
  --cpu 1 --memory 2Gi --replica-timeout 7200 --replica-retry-limit 1 \
  --parallelism 1 --replica-completion-count 1 \
  --registry-server lrnregistry.azurecr.io --mi-system-assigned
```

Cron runs in UTC: 0 22 * * * is 06:00 Singapore time (UTC+8).

7. RunId design with Container Apps Jobs

One job execution = one LRN RunId. Store the Azure execution name next to the RunId, so any LRN log row can be traced to its Azure execution and its container logs.

flowchart LR

```

A[Job execution starts] --> B[Create RunId<br/>LRN_RUN_Log = Running]
B --> C[Per lab: steps<br/>LRN_Step_Log]
C --> D[Staging load + swap<br/>per lab]
D --> E[Lab success row<br/>per RunId]
E --> F[RunId = Success<br/>exit 0]
C -. error .-> G[LRN_Error_Log<br/>RunId = Failed, exit 1]

```

Concern	Design
Creating the RunId	Generate it at program start, before any lab is processed, and insert the LRN_RUN_Log row with status Running
Link to Azure	Read the execution name from the container environment (Container Apps sets CONTAINER_APP_JOB_EXECUTION_NAME; confirm in the first test run) and save it in a new column, e.g. ExecutionName
Per-lab tracking	Keep LRN_Step_Log and the per-lab success row per RunId; FileLogId, RowHash and audit columns stay unchanged
ReportRunIdInfoLog	Write RunId, ReportType, SourceSystem = the job name, CreatedBy = the managed identity or job name (no interactive user in a container)
Failure	Log to LRN_Error_Log, mark the RunId Failed, exit non-zero so Azure shows the execution as Failed and alerts fire
Automatic retry	A retry restarts the program. On start, close any Running RunId with the same ExecutionName as Failed, then create a new RunId. Staging-and-swap makes the reload safe
Manual re-run	Start the job from portal or CLI with an override such as LAB_FILTER or WEEK_FOLDER; it gets its own RunId and ExecutionName
Overlap protection	sp_getapplock taken right after the RunId is created; if the lock is held, log Skipped and exit 0

Concern	Design
Timeout	If Azure kills the container at replica timeout, the RunId stays Running; the next run's startup check marks it Failed (TimedOut)

```
# manual re-run for one lab
az containerapp job start --name lrn-master-file-processor --resource-group
rg-lrn \
  --env-vars LAB_FILTER=NWL WEEK_FOLDER=2026-W38
```

The env override flags shown above must be added to the worker; they are a proposal, not an existing feature.

8. Constraints and corrections to plan for

These points were missing or unclear in the earlier notes, and each one changes design or budget.

Topic	What to know	Action
Linux only on Container Apps	No Windows containers; .NET Framework apps cannot move there without porting	Build Linux images; LRN apps are .NET Core and Python, so this is feasible
Consumption memory limit	Max 4 vCPU / 8 GiB per replica; a ~10 GB Python job will not fit	Reduce memory (chunking, streaming) or use a memory-optimised Dedicated workload profile
Reaching SQL MI	MI lives inside a VNet	VNet-integrated Container Apps environment in a subnet routed to the MI subnet; avoid the MI public endpoint
Private endpoints	Add a Dedicated plan management charge even on Consumption; private ACR needs Premium	Decide the network model early

Topic	What to know	Action
Functions time limits	Plan-dependent run limits; HTTP about 230 s	No long batch programs in Functions without redesign
Schedules in UTC	Task Scheduler uses local time	Convert all schedules (Singapore = UTC+8)
Overlapping runs	A new execution can start while the last one runs	Parallelism 1 + database lock (see section 7)
Outbound IP	Container outbound IPs can change	NAT Gateway if a firewall must allow-list LRN
Double processing during cut-over	VM copy and Azure copy both running	Disable the VM Task Scheduler entry or service at go-live; test Azure against a test DB first
Python service shape	Earlier notes called it both continuous and scheduled	Treated here as scheduled (Task Scheduler); if it is really a loop, use a Container App instead

9. Cost estimation

Indicative total: roughly USD 72–244 per month for the app tier, excluding Azure SQL MI. All rates are approximate US-region pay-as-you-go figures; Southeast Asia is usually slightly higher, so confirm in the Azure Pricing Calculator. SGD is roughly USD × 1.3.

9.1 How each service is billed

Service	Billing meter	Approximate rate (USD)
Container Apps / Jobs (Consumption)	vCPU-s + GiB-s while running, requests	vCPU ≈ 0.000024/s; memory ≈ 0.000003/GiB-s. Free per subscription per month: 180,000 vCPU-s, 360,000 GiB-s, 2M requests (billing docs)
Container Apps Dedicated profile	Per node while running + fixed management charge	Management fee on the order of 70+/month; node price by size

Service	Billing meter	Approximate rate (USD)
App Service	Per plan instance-hour	Basic B1 Linux $\approx$ 13/month; Premium v3 from $\approx$ 60/month; Windows costs more
Functions (Flex Consumption)	Executions + memory-time	Free grant; small handlers usually a few dollars
Container Registry	Fixed per day by tier	Basic $\approx$ 5, Standard $\approx$ 20, Premium $\approx$ 50 per month
Key Vault	Per 10,000 operations	$\approx$ 0.03; usually under 5/month
Log Analytics	Per GB ingested	$\approx$ 2.30–2.80 per GB
NAT Gateway (optional)	Per hour + GB	$\approx$ 32/month + data
Private endpoint (optional)	Per hour + GB	$\approx$ 7/month each + data

9.2 Jobs: cost per hour of run time

```
\text{cost} = \text{vCPU} \times \text{seconds} \times 0.000024 + \text{GiB} \times \text{seconds} \times 0.000003
```

Job size	USD per run-hour	USD for 60 run-hours / month
0.5 vCPU / 1 GiB	0.054	3.24
1 vCPU / 2 GiB	0.108	6.48
2 vCPU / 4 GiB	0.216	12.96
4 vCPU / 8 GiB (Consumption max)	0.432	25.92

Run-hours per month = runs per month $\times$ minutes per run $\div$ 60. Example: 2 vCPU / 4 GiB, 45 min every hour = 540 run-hours $\approx$ USD 117 ($\approx$ 111 after the free grant). The same job at 10 min every 30 min = 240 run-hours $\approx$ USD 52. Run length and memory drive the cost most.

9.3 Monthly estimate for LRN (excluding SQL MI)

Assumes ReportsApi and web app at 0.5 vCPU / 1 GiB with one replica each, Master File Processor 1 vCPU / 2 GiB for ~30 run-hours, Python within 8 GiB for ~60 run-hours, Standard ACR.

Component	Low (USD)	High (USD)
LRN.ReportsApi (Container Apps, min 1)	15	40
.NET web app (Container Apps)	15	40
React UI (Static Web Apps)	0	9
Python processor (Job)	13	60
Master File Processor (Job)	3	15
Other scheduled workers (Jobs)	0	20
Container Registry (Standard)	20	20
Key Vault	1	5
Log Analytics / App Insights	10	40
Monthly free grant	-5	-5
Subtotal	≈ 72	≈ 244
Optional: Dedicated profile for a >8 GiB job	+75	+200
Optional: NAT Gateway / private endpoints	+32	+60

Alternative web tier: one Linux App Service Plan (B2 or P0v3) for both .NET apps is about USD 25-80/month, paid 24/7. Azure SQL MI stays the largest cost and is unchanged by this move.

10. Migration roadmap

Move one workload at a time, starting with LRN.ReportsApi to learn the pipeline.

Track the work in [Migration to-do and checklist](#) .

Phase	Work	Done when
0. Foundation	Resource group, VNet subnet routed to SQL MI, VNet-integrated Container Apps environment, ACR, Log Analytics; grant Managed Identities access to ACR, Key Vault, SQL	A test container can query SQL MI
1. LRN.ReportsApi	Build Linux image, docker run locally, test in Postman, push to ACR, deploy to Container Apps, secrets to Key Vault	Same results from Azure as from IIS
2. Python processor	Containerise, measure peak memory, cut it below 8 GiB if possible, create scheduled Job on test data	Successful executions; Task Scheduler entry disabled
3. Master File Processor	Run-once console host, Linux path check, RunId changes from section 7, scheduled Job with parallelism 1	RunId logs complete for all enabled labs; VM service stopped
4. Other services + web app	Classify each as Job, Container App or Function; migrate one by one	Nothing left on the VM
5. Retire VM	Monitor 2-4 weeks, downsize, decommission	VM cost removed

Automate build and deploy per repo with GitHub Actions: build, tag with version, push to ACR, update the app or job.

11. Information needed for a firm estimate

Answer these per workload to turn the ranges above into a Southeast Asia estimate in SGD.

- ☐ Runs once and exits, or stays running?
- ☐ Current Task Scheduler interval

- ☐ Typical and worst-case run time
- ☐ Peak memory and CPU
- ☐ File volumes and temp disk use
- ☐ API traffic pattern and acceptable cold start (min replicas 0 or 1)
- ☐ Network rules for SQL MI and SharePoint (VNet, private endpoints, NAT)
- ☐ Log volume per day
- ☐ Current VM size and monthly cost